Python for computer vision

1 Basic Python

(a) **Libraries**: Libraries can be imported using the keyword `import`. After importing a library, its functions can be used by prepending them with the library name (e.g. `np.array()`). Aliases (to reduce typing) can also be used for renaming a library or module using the keyword `as`.

```
import cv2 # import entire library
import numpy as np # import library as a specific, shorter name
from matplotlib import pyplot as plt # import a specific module
↪  from a library
```

(b) **Basic data types**: Python includes basic data types, such as integer, float, boolean, and string. The type does not need to be explicitly declared, and many basic operations are defined in the interpreter.

```
x = 3 # defines an integer
print(type(x))
x+=1 # adds 1
x*=2 # multiplies by 2
y = 2.5 # defines a float
print(type(y))
str1 = "Hello" # strings can be defined with double
str2 = 'world!' # or single quotes
print(str1+" "+str2)
boolean1 = True # booleans are defined with a capital letter
boolean2 = False
print(boolean1 and boolean2) # logical operations are implemented
↪  with words (and, or, not)
```

(c) **Printing values**: Printing the values of variables can be useful for tracking progress or debugging. Objects implement their own print functions so you are able to use the function `print()` to display numpy arrays directly in the output. More complex formatting can be performed using python's f-strings which can include variable names that are converted to their string representations and displayed at runtime. An example showing various ways of printing a floating point number is shown below.

```python
import numpy as np
a = np.sqrt(2)

print(a) # displays the value
print(f'Value: {a}') # displays the value in a string
print(f'{a:0.3f}') # limits the number of decimal places
print(f'{a=}') # displays the name and the value at the same time
print(f"{a=:.3f}") # combines = and formatting in one line
print(f"{a+1=}") # entire expressions can be printed in the same
↪ way
```

(d) **Lists and tuples**: Tuples are immutable (their elements can be accessed, but they cannot be changed), while lists are mutable and support sorting and other list operations. Both are indexed using brackets (`[]`). You can obtain the size of most python objects using the function `len()`.

```python
a = (1,2,3) # tuple
b = [1,2,3,4] # list
print(len(a))
print(len(b))
b[0]=9 # set an element at index 0
b.append(5) # add a new element to the end of the list
print(len(b))
a[1]=1 # this does not work because a is a tuple
```

(e) **Dictionaries**: Dictionaries are python's associative arrays, whose values are accessed using keys. Integers, tuples, and strings can be used as dictionary keys. The dictionary values are accessed using brackets (`[]`). You can iterate over the keys and values of a dictionary using `.items()`.

```python
c = {'d': 15, 'e': 'f'}
print(c['d']) # prints 15
for key, value in c.items():
    print(key, value)
```

(f) **Indentation**: Python uses indentation to separate blocks of code, such as functions, loops and if statements. Indentation can be either tabs or spaces, but the indentation needs to be consistent. Semicolons can be used if several statements appear on the same line, but the PEP 8 [1] standard generally discourages their use.

(g) **If statements**: Branching works with if statements, where the conditions directly follows the keyword `if`, and is ended by a colon. The body of the statement has to be indented. The structure can be extended with keyword `elif` which signifies alternative conditions and optionally finished with the keyword `else`. Additionally, `if` can also be used as a ternary conditional operator and evaluated to a value in one line.

[1] PEP 8

```
a = True
if a:
    print("true")
else:
    print("false")

result = True if a else False
print(result) # prints True
```

(h) **For loops**: The core of the for loop must be indented in python. Python can iterate over many different objects (they are called *iterables*) like tuples and lists. Explicit indexing (as in C or Java) is usually not needed. Note that the indent can be either the tab character or a number of spaces (usually 4 per indent level). These can be used interchangeably in a script, but not within a function.

```
a = [1,5,4,2,3,4,5,6,7,8,1]
for x in a: # iterates over elements directly
    print(x) # this must be indented
```

If you however need an index, the simplest way of getting both at the same time is using `enumerate`

```
a = [1,5,4,2,3,4,5,6,7,8,1]
for i, x in enumerate(a): # iterates over elements using an
↪  enumerator object
    print(i,x) # displays both the index and the corresponding
    ↪  value
```

(i) **Function definition**: Functions are defined by the keyword `def`. Function arguments are defined in parentheses. Python will accept arguments of any type, and you need to make sure to pass and use the objects of the appropriate type (or check the type within the function yourself).

```
def sum(a,b):
    return a+b
```

You can optionally specify the type of the parameters and the data type the function returns, but python will not complain if the argument types don't match. This is mostly used for static code analysis and for documentation purposes.

```
def product(a: int, b: float) -> float:
    return a+b
```

Anonymous functions are also supported using the keyword `lambda`. Python also supports higher-order functions, meaning that it is possible to pass functions as objects, using them as arguments and returning them from functions.

```
mysum = lambda x, y: x+y # defines an anonymous function with 2
↪  arguments
print(mysum(1,2)) # prints 3

applyFunction = lambda fun, x: fun(x)
print(applyFunction(math.sqrt, 2)) # prints the square root of 2

# returns a function that calculates the nth power of the input
def myPower(n):
    return lambda x: x**n
squareFun = myPower(2)
print(squareFun(2)) # prints 4
cubeFun = myPower(3)
print(cubeFun(2)) # prints 8
```

(j) **Class definition**: Python supports defining new classes using the keyword `class`. The attributes of the class instance can be accessed from within the class using the keyword `self`.

```
class Greeter(object):

    # Constructor
    def __init__(self, name):
        self.name = name  # Create an instance variable

    # Instance method
    def greet(self, loud=False):
        if loud:
            print(f'HELLO, {self.name.upper()}!')
        else:
            print(f'Hello, {self.name}')

g = Greeter('Fred')  # Construct an instance of the Greeter class
g.greet()            # Call an instance method; prints "Hello,
↪  Fred"
g.greet(loud=True)   # Call an instance method; prints "HELLO,
↪  FRED!"
```

(k) **Unpacking**: Python supports unpacking a tuple into named variables. If you know the number of tuple elements, you can assign a variable name to each one. This is especially useful when a function returns more than one value.

```
def myfunction(a,b):
    return a+b, a*b, a**b

my_sum, my_product, my_power = myfunction(2,3) # unpack the result
↪  immediately
```

(l) **Indexing**: Elements of iterable objects (such as tuples or lists) can be accessed using brackets. The basic functionality of accessing one element (e.g. `a[2]`) is actually shorthand for more complex indexing using colon `:`. The full indexing syntax is `iterable[start:end:step]`, where `start` and `end` signify the first and the last index. These indexing statements create a new array. Negative indices are supported for all the indexing arguments, and they wrap around the array. Accessing the last element is performed by `a[-1]`. Note that the wrap around is only defined once (i.e. for an array of length n, the valid indices are on the interval `[-n,n-1]`).

```
a = [0,1,2,3,4,5]
a[2] # 2
a[3:] # [3,4,5]
a[-2:] # [4,5]
```

(m) **For loops in one line**: Sometimes it is cleaner to do some simple operations in one line. Ternary operators are also supported in the same way:

```
a = np.arange(10) # list of all integers from 0 to 9
[x**2 for x in a] # squaring every element of the list
[x**2 if x % 2 == 0 else x for x in a] # squaring only even
↪ elements
[x for x in a if x % 2 == 0 and x % 3 == 0] # extracting elements
↪ divisible by 2 and 3
```

2 Numpy and array iteration

(a) **Numpy**: Numpy is a numerical computational library. It supports multidimensional arrays and functions to manipulate them. The arrays are defined by their type and their size.

```
a = np.array([[1,2],[3,4]]) # create a numpy array from nested
↪ lists
print(a.shape, a.dtype) # (2,2) int64
print(a[1,1]) # 4
```

(b) **Data types**: The default integer data type in numpy is `np.int64`, while for floating point numbers it is `np.float64`. Type conversion can be performed using the function `np.astype()`.

(c) **Initialization**: Numpy includes some helper functions to aid with data initialization.

```
z = np.zeros((2,5)) # creates a new array of zeros
o = np.zeros((3,5)) # creates a new array of ones
o_f = np.zeros((2,2), dtype=np.float32) # we can explicitly define
↪ the type at initialization
i = np.eye(3) # creates a 3x3 identity matrix
```

```
c = np.ones_like(z) # creates an array of ones the size of z
↪  (equivalent to np.ones(z.shape))
r = np.random.random((2,2)) # creates a 2x2 matrix with random
↪  values
```

(d) **Array indexing**:

Arrays are indexed by their dimensions using brackets (`[]`). Multidimensional arrays can be indexed using multiple indices in sequence. Each of them uses the standard python indexing syntax and can contain more complex indexing patterns. For example, `I[0:100, 0::2]` returns the first 100 rows and every second column from a 2-D array.

(e) **Iterating over multidimensional arrays**:

This can be done with nested `for` loops, but due to increasing time complexity (and python's loops being relatively slow) it should be avoided if at all possible.

```
# Open an image and convert it to Numpy array.
I = cv2.imread('image.jpg')

for i in np.arange(I.shape[0]):
    for j in np.arange(I.shape[1]):
        pixel_value = I[i,j]
```

(f) **Reshaping arrays**:

Sometimes, the spatial component of the data is not as important. In these cases, arrays can be unrolled into 1-D vectors. Arrays can also be reshaped using the function `np.reshape()` as long as the number of elements remains the same. The transpose of an array can be accessed using `np.transpose()`. It should be noted that one-dimensional arrays do not have a transpose (which is reasonable), but numpy will not refuse the operation but instead return the same array (this is less reasonable).

```
a = np.random.rand(4,6) # shape: (4,6)
b = a.reshape(-1) # shape: (24,)
c = a.reshape((12,2)) # shape: (12,2)
d = a.transpose() # shape (6,4)
a.T # equivalent to d
np.transpose(a) # equivalent to d
```

(g) **Accessing specific elements of an array**:

Sometimes it can be useful to only access a small subset of array elements based on some condition. This can be done by creating a boolean array that will serve as an index for the wanted elements. The selected elements can be extracted or replaced in the same way.

```
a = np.random.randint(0,100, (10,10)) # array of random integers
↪ from interval [0,100]
mask = a%13==0 # boolean mask of the conditional
res = a[mask] # array of all array elements marked by the mask
↪ array (all elements divisible by 13)

a[mask]=a[mask]**3 # cubing the selected elements and replacing
↪ them in the original array
```

(h) **Transposing an array**: In numpy, vectors with a single dimension cannot be transposed. This can sometimes cause unexpected behavior. In case you need the vector to be transposable, you should use `np.expand_dims()` to add a new dimension.

```
k = np.array([1, 2, 3], dtype=float)
print(k)  # [1. 2. 3.]
print(k.T)  # The same result, [1. 2. 3.]
k = np.expand_dims(k, 0)  # array needs to be 2D to support
↪ transposing
print(k)  # [[1. 2. 3.]]
print(k.T)  # [[1.], [2.], [3.]]
```

(i) **Array arithmetic**: Most standard arithmetic operations work as expected in numpy (addition, subtraction, multiplication). However, sometimes numpy does something unexpected based on data types and syntax. We present some common cases below.

```
a = np.random.randint(0, 10, (5,5)).astype(np.uint8)
print(a-10) # wrap-around to 255
print((a/2).dtype) # type coercion to np.float64

b = np.random.randint(0, 10, (2,2))
c = np.array([[1,2], [3,4]])
print(b*c) # per-element multiplication
print(c@c) # matrix multiplication (equivalent are np.matmul() and
↪ np.dot())

d = np.random.randint(0, 10, (2,1)) # column vector
print(c*d) # treats every element of d as a factor for every row of
↪ c
```

(j) **Helpful functions**: Numpy also implements many useful functions that you can use to analyze arrays, such as `np.min()`, `np.max()`, `np.sum()`, `np.mean()`, etc. Below, we present some basic examples.

```
x = np.array([[1, 2], [3, 4]], dtype=np.float64)
print(np.sum(x)) # sums up all the elements
print(np.sum(x, axis=0)) # sums up the columns
print(np.sum(x, axis=1)) # sums up the rows
```

```
y = np.array([[1, 2], [3, np.nan]])
print(np.mean(y)) # returns np.nan if any value is NaN
print(np.nanmean(y)) # correctly ignores NaN
```

3 Plotting with pyplot

(a) **Subplots**:

Subplots are used to display multiple images in the same windows. The first two arguments define the grid shape (e.g. `plt.subplot(2,3,x)` defines a 2 row, 3 column grid) and the last one is used to focus on a specific subplot cell. This means that after calling `plt.subplot` with a specific index, the next `plt.imshow` call will be displayed in the currently active grid cell. Note that the cells are linearly indexed and the indices start with 1.

```
# Open an image and convert it to Numpy array.
I = cv2.imread('image.jpg')

plt.clf() # clears the figure (is necessary if plotting many images
↪  in a sequence)

plt.subplot(1,2,1) # creates two columns and focuses on the left
↪  one
plt.imshow(I)
plt.title('RGB image') # displays a title for the subplot cell

plt.subplot(1,2,2)
plt.imshow(I[...,0])
plt.title('blue channel')

plt.show()
```

(b) **Animations**: You can display the intermediate results of an algorithm or loop in a continuous but non-interactive way by displaying the figure with a non-blocking command.

```
import matplotlib.pyplot as plt
import numpy as np

N = 100 # number of points
x = np.linspace(0, 3*np.pi, N) # values
offset = 0.1

while True:
    x+=offset
    y = np.sin(x)

    plt.clf() # clears the figure
```

```
    plt.plot(x,y,'r-') # displays the curve
    plt.draw(); plt.pause(0.01) # draws and displays the plot
```

(c) **Sliders**: A more involved but interactive way of displaying, for example, the effect of algorithm parameters on your image is using sliders.

```
import matplotlib.pyplot as plt
import numpy as np
from matplotlib.widgets import Slider

# Function to apply threshold: keep original intensities above
↪  threshold, set to zero otherwise
def apply_threshold(data, thresh):
    return np.where(data > thresh, data, 0)

# Generate a 2D Gaussian blob
height = 200
width = 200
x = np.linspace(-3, 3, width)
y = np.linspace(-3, 3, height)
X, Y = np.meshgrid(x, y)
Z = np.exp(-(X**2 + Y**2))

# Initial threshold value
thresh0 = 0.5

# Create the figure and axes
fig, ax = plt.subplots()
plt.subplots_adjust(bottom=0.25)
im = ax.imshow(apply_threshold(Z, thresh0), cmap='gray')
ax.set_title('Thresholded Gaussian Blob')

# Slider axis
ax_thresh = plt.axes([0.25, 0.1, 0.65, 0.03])
slider = Slider(ax=ax_thresh, label='Threshold', valmin=0.0,
↪  valmax=1.0, valinit=thresh0)

# Update function
def update(val):
    thresh = slider.val
    im.set_data(apply_threshold(Z, thresh))
    fig.canvas.draw_idle()

# Connect slider and update
slider.on_changed(update)
plt.show()
```

4 Debugging image processing algorithms

While developing image-based algorithms, there are several ways that mistakes can be made. This section is meant to briefly explain the most common issues and how to address them. A reminder of these will also be shown at the top of each assignment.

4.1 Axes

The order of the axes for determining a specific pixel is a convention. In OpenCV and Numpy, the order is (H, W, C), which means that the first index addresses the rows (from the top of the image), the second addresses the columns, and the last one addresses the (color) channel. In the following example, we first set a pixel to 1, then try to plot a red dot on the same pixel. It turns out that the functions use different interpretations of the coordinate systems, thus producing an unexpected result. Function `plt.imshow()` displays the image with the same axis order as numpy, whereas the function `plt.plot()` interprets the coordinates as the standard cartesian coordinates, first moving along x horizontally, then along y vertically (note that when the image is displayed, the y axis is pointing downwards).

```
N = 10
im = np.zeros((N,N))

x = 1
y = 2

im[x,y] = 1

plt.clf()
plt.imshow(im)
plt.plot(x,y,'r.')
plt.show()
```

Such behavior can cause long-lasting problems while developing algorithms if unchecked. The main advice here is to always check the assumptions about the axes by drawing or otherwise verifying the actual interpretation of the data.

4.2 Data range and type

Image-based algorithms can have different assumptions about the representation and range of the input data. Some algorithms explicitly work on images represented with 8-bit unsigned integers (commonly used as `np.uint8`), or their 3-channel variants. Other algorithms work with images consisting of floating point pixels (commonly `np.float32` or `np.float64`). Using the incorrect data type can lead to various problems, including fully black or white images, errors when using division or some other mistake.

An additional problem can arise with data ranges. Some algorithms assume the pixel values are normalized (i.e. range from 0 to 1), while other algorithms only use floating point values without any assumptions regarding the range of the values. It is helpful to

check the type and range the algorithm expects, as well as checking your input data to ensure the results are valid. You can use functions like `np.min()`, `np.max()`, `np.unique()`, and `data.dtype` for this.

Another issue can occur when displaying image data. Since (most) monitors use 8-bit RGB pixels, not all image data can be displayed. This forces the software solutions to preprocess your data before displaying it. If the drawing software does this in an unexpected way, your results can look bad despite being correct, or very different images can appear similar on the screen. In the following example, the images will appear the same, despite being very different in type and values. This is because `plt.imshow()` tries to normalize the images so they are always clearly visible. This is often useful, but can cause problems when inspecting the intermediate results of algorithms. Note that `pyplot` also uses a blue-yellow colormap by default, which you can change using the argument `cmap='gray'`.

```
import numpy as np
import matplotlib.pyplot as plt

N = 5
x = y = N//2

im1 = np.zeros((N,N), dtype=np.uint8)
im2 = np.zeros((N,N), dtype=np.uint8)
im3 = np.zeros((N,N), dtype=np.float32)
im4 = np.zeros((N,N), dtype=np.float32)

im1[x, y]=1
im2[x, y]=255
im3[x, y]=1e-3
im4[x, y]=1e3

plt.subplot(2,2,1)
plt.imshow(im1, cmap='gray')
plt.subplot(2,2,2)
plt.imshow(im2, cmap='gray')
plt.subplot(2,2,3)
plt.imshow(im3, cmap='gray')
plt.subplot(2,2,4)
plt.imshow(im4, cmap='gray')
plt.show()
```

4.3 Synthetic test cases

Images are generally large matrices, ranging into many thousands of pixels. While most image processing or computer vision algorithms are eventually run on full-size images, there is no reason that you should develop algorithms on very large images. If your only task is to program a simple filter or procedure, its steps and results can be easier to observe on a smaller image.

The practice of generating synthetic images can simplify the development and debugging process by a lot. If, for example, you are developing an edge detection algorithm, create an image with a very distinct edge, and run your algorithm on it. If anything went wrong, determining the source of the problem can be easier observed on simple data.

Another important thing is being able to inspect the actual values of pixels. This is available by hovering your mouse over the figure shown by `plt.imshow()`, displaying the coordinates and corresponding values in the window corner (top left on linux). You are encouraged to use this functionality to help with debugging your code. This is also the main reason using Jupyter notebooks is discouraged for this course. By default, Jupyter embeds the plots inline, which precludes the use of interactive inspection. You can force an interactive window using the command `%matplotlib qt` inside a Jupyter notebook if you wish.

Using several subplots to display the intermediate steps of the algorithm will allow you to check if each step did the appropriate changes. It can also help with the assignment defense. Implementing sliders or animations, while time-consuming, can additionally give you better insight into how different parameter values affect the algorithm results.